SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

Hospital Management System

Team 14
Version 1.0
16-09-2026

Item	Details
Project	Hospital Management System
Team Number	14
Technology	C / C++
Reference	Hospital Management System SRS v1.0
Status	Draft / Mini-Project Submission

1. Introduction

This document describes the architecture and basic software design of the Hospital Management System. It is based on the existing SRS and keeps the design suitable for a small C/C++ desktop application.

1.1 Purpose

The purpose is to show how the HMS requirements are divided into components, how those components interact, and how the main functions handle normal and error conditions.

1.2 Scope

The design covers authentication, patient management, doctor management, appointments, billing, reports, administration, storage, input validation and role-based access. Advanced equipment and external insurance integration are not included.

2. Architecture Overview

2.1 Architectural Pattern

The proposed architecture uses a simple Layered Architecture. This fits the C/C++ desktop application described in the SRS and keeps user interaction, business rules and storage separate.

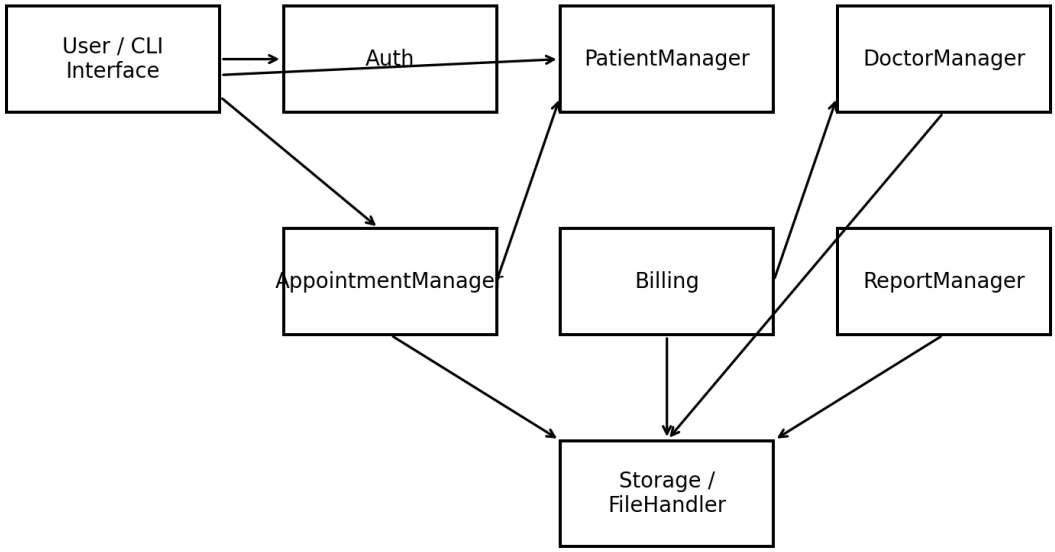
Layer	Main Responsibility	Examples
Presentation / CLI Layer	Menus, prompts, input and output	Login menu, patient menu, appointment menu
Business Logic Layer	Validation and application rules	Auth, PatientManager, DoctorManager, AppointmentManager, Billing, Reports
Data Access / Storage Layer	Read/write persistent data	Storage / FileHandler, local files or database

2.2 Main Flow

- The user interacts with the CLI/menu layer.
- The selected operation is passed to the relevant business component.
- Business components validate inputs and enforce rules such as role access and double-booking prevention.
- The Storage/FileHandler reads or writes the required patient, doctor, appointment, billing or user data.
- The result is returned to the menu layer and displayed to the user.

3. Component Diagram & Description

HMS Component Diagram



Component	Description
Auth	Handles username/password login and identifies the user's role.
PatientManager	Registers, updates and searches patient records.
DoctorManager	Adds/updates doctor information and checks doctor availability.
AppointmentManager	Creates, cancels and reschedules appointments and prevents double booking.
Billing	Stores basic billing information for a patient visit.
ReportManager	Generates simple patient and appointment reports.
Storage / FileHandler	Provides a common interface for reading and writing persistent data.
User / CLI Interface	Provides simple menus and collects user input.

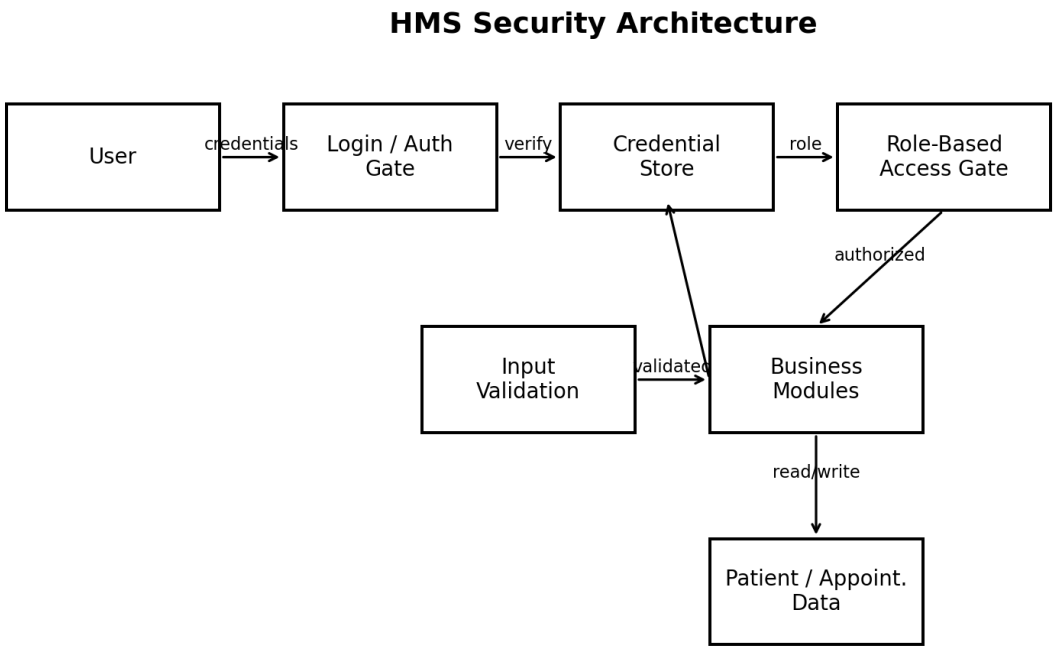
4. Requirements Traceability

The architecture maps the SRS functional requirements to the components responsible for implementing them.

Component	SRS Requirements
Auth	HMS-F-001
PatientManager	HMS-F-002, HMS-F-003, HMS-F-004
DoctorManager	HMS-F-005, HMS-F-006, HMS-F-007
AppointmentManager	HMS-F-008, HMS-F-009, HMS-F-010, HMS-F-011, HMS-F-012
Billing	HMS-F-013
ReportManager	HMS-F-014
Auth + User Administration	HMS-F-015

The non-functional and security requirements are handled across the layers. Performance and usability are mainly supported by the presentation and business layers; reliability and data integrity are supported by the storage layer; authentication, authorization and validation are handled by Auth, role checks and validation logic.

5. Security Architecture



Security is applied before protected business operations are executed. The design uses authentication, role-based authorization and input validation as separate checks.

- Authentication gate: protected functions require a valid username/password.
- Credential storage: passwords should be stored as hashes rather than plaintext in the credential store.
- Role gate: after login, the user's role determines which menus and protected functions can be used.
- Input validation: patient, doctor and appointment inputs are checked before storage.
- Restricted records: patient information is returned only after the relevant access check passes.
- Administrative actions: important actions such as user creation and record deletion are recorded as required by the SRS.

6. Detailed Design

6.1 Core Data Structures

Structure	Important Fields
Patient	int patientID; string name; string contact; string address;
Doctor	int doctorID; string name; string specialization; bool available;
Slot	string date; string time;
Appointment	int appointmentID; int patientID; int doctorID; Slot slot; string status;
User	string username; string passwordHash; string role;
Bill	int billID; int patientID; double amount; string status;

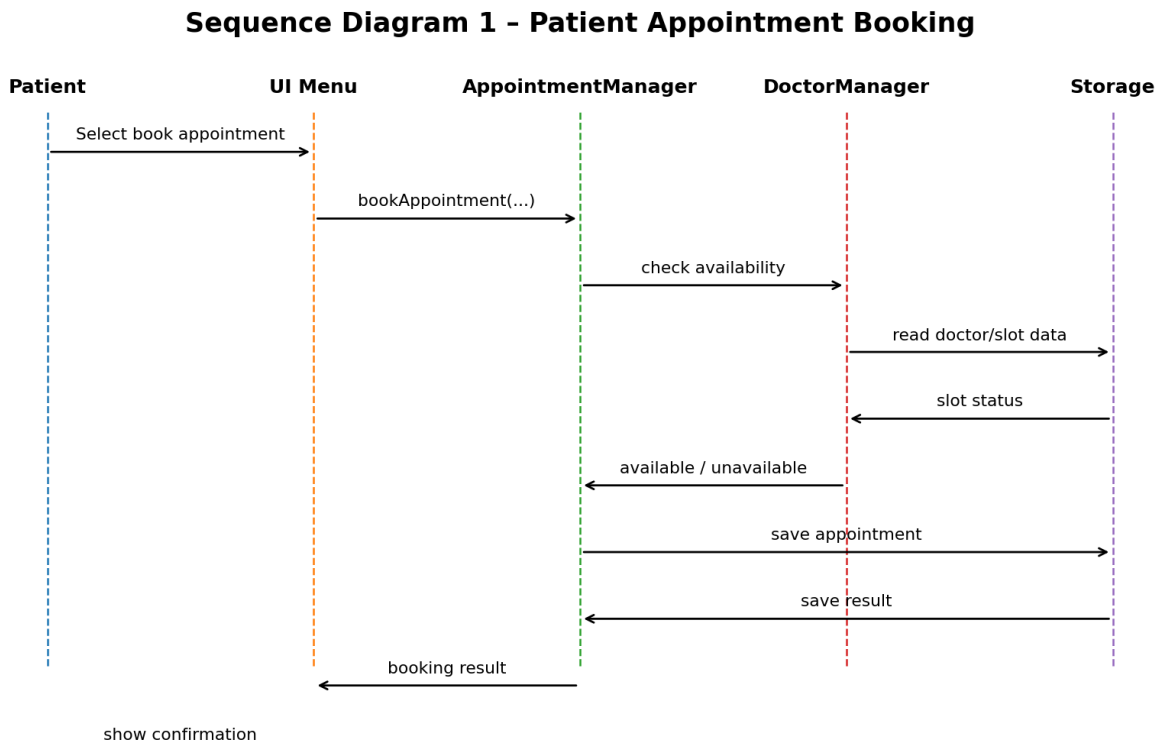
6.2 Core Function / API Design

Function	Parameters	Return	Purpose

bool authenticate(...)	username, password	bool	Checks credentials and allows/denies login.
Patient registerPatient(...)	patient details	Patient	Creates a patient record with a unique ID.
Patient searchPatientByID(...)	int id	Patient / not found	Finds a patient by ID.
bool updatePatient(...)	id, patient details	bool	Updates an existing patient.
Doctor addDoctor(...)	doctor details	Doctor	Adds a doctor record.
bool updateDoctor(...)	id, doctor details	bool	Updates doctor details.
bool isSlotAvailable(...)	doctorID, Slot	bool	Checks whether a doctor/time slot is free.
bool bookAppointment(...)	patientID, doctorID, Slot	bool	Creates an appointment if the slot is available.
bool cancelAppointment(...)	appointmentID	bool	Cancels an existing appointment.
bool rescheduleAppointment(...)	appointmentID, Slot	bool	Moves an appointment to a valid free slot.
bool saveBill(...)	Bill bill	bool	Stores basic billing information.
bool createUser(...)	User user	bool	Creates an administrative user account.
bool disableUser(...)	username	bool	Disables an existing account.
Report generateReport(...)	report type, filters	Report	Creates a simple report.

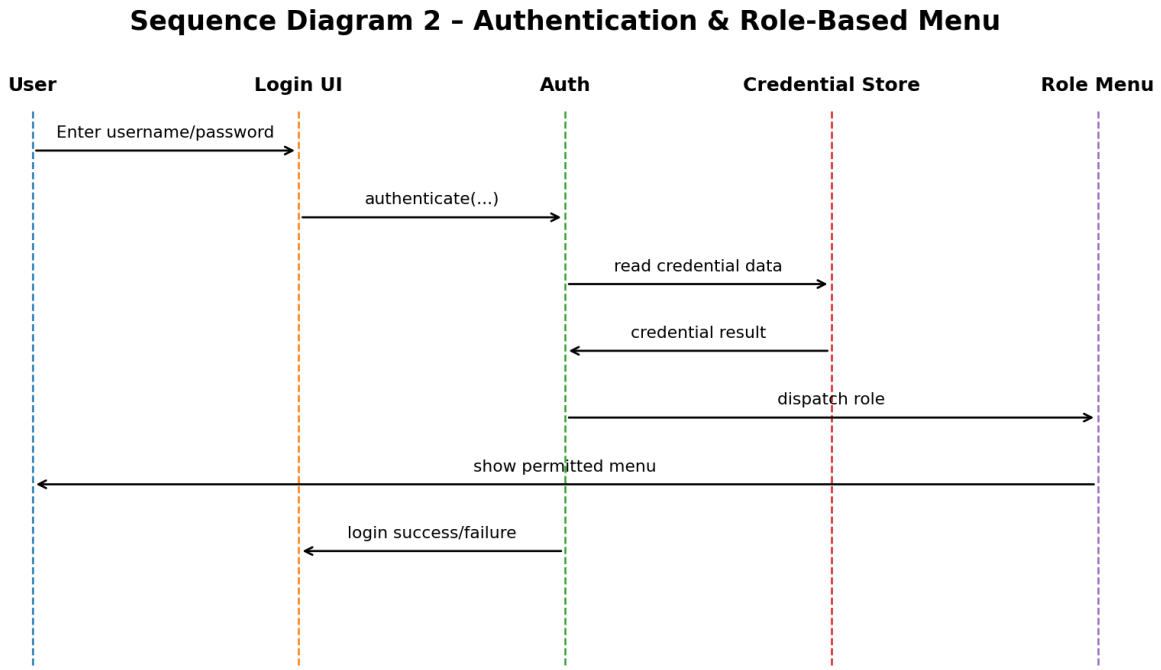
7. UML Sequence Diagrams

7.1 Patient Appointment Booking



The booking flow checks doctor availability before saving the appointment. This supports HMS-F-008 and HMS-F-009.

7.2 User Authentication & Role-Based Menu Dispatch



The login flow verifies credentials, determines the user's role and then dispatches the permitted menu. This supports HMS-F-001 and HMS-SR-001/HMS-SR-002.

8. Error Handling

Situation	Handling	Result
Empty required input	Validate before calling the business function.	Show a simple input error and ask for the value again.
Invalid patient/doctor ID	Check whether the record exists.	Return not-found error; do not modify data.
Invalid date/time	Validate format and required fields.	Reject the appointment input.
Double booking	Check the doctor/time slot before saving.	Reject the second booking.
Unauthorized role	Check role before protected operation.	Deny access and return to the permitted menu.
Duplicate patient/user record	Check unique identifier/username.	Reject duplicate and show an error.
File/database read failure	Return storage error; do not continue with corrupted/unknown data.	Show error and allow retry/restart.
File/database write failure	Return failure status and do not report a successful save.	Show error and preserve the previous valid state where possible.

Suggested simple error values: OK = 0, INVALID_INPUT = 1, NOT_FOUND = 2, UNAUTHORIZED = 3, DOUBLE_BOOKING = 4, DUPLICATE = 5, STORAGE_ERROR = 6.

9. Data Access & Storage Design

The SRS permits local files or a database. The design keeps Storage/FileHandler independent of the business modules so the storage method can be changed without rewriting the main application logic.

- Patient data is stored and retrieved through Storage/FileHandler.

- Doctor and appointment data use the same storage interface.
- Billing and user-account information are also persisted through the storage layer.
- After application restart, valid stored records should be loaded correctly to satisfy the data-integrity requirement.

10. Design Traceability to Security & Quality Requirements

Requirement	Design Support
HMS-NF-001	Keep operations local and lightweight; avoid unnecessary repeated file operations.
HMS-NF-002	Use simple, stable local storage and handle storage failures.
HMS-NF-003	Role checks and authentication protect patient information.
HMS-NF-004	Validate data before saving and verify stored data after restart.
HMS-NF-005	Use simple menus, clear prompts and short error messages.
HMS-SR-001	Auth component requires username/password for protected functions.
HMS-SR-002	Role-Based Access Gate dispatches permitted menus/functions.
HMS-SR-003	Patient records are returned only after authorization.
HMS-SR-004	Validation layer checks data before storage.
HMS-SR-005	Admin actions are recorded in the relevant administration/storage logic.

11. Assumptions & Design Constraints

- The application is a small C/C++ desktop system.
- The basic version does not require network communication.
- Storage may be local files or a database, as allowed by the SRS.
- Only dummy/test patient information is used during development and testing.
- The design is intentionally simple so it can be implemented and demonstrated within the mini-project.

12. Approval

Role	Name	Signature / Date
Team Member	Nirav Choudhary	
Team Member	Pranav Kaushal	
Team Member	OP Medha	
Team Member	Pranav Swaroop	